

DevOps Complete Guide

Basic to Advanced: concepts, commands and code snippets

Linux | Git | Cloud (AWS, Azure, GCP) | Docker | Kubernetes | Terraform | Ansible | Jenkins | CI/CD | Helm |
Monitoring | GitOps | DevSecOps

Each chapter goes from first principles to advanced usage, with comparison tables, ready-to-run snippets, common mistakes and interview-style questions.

Contents

7. Terraform: Infrastructure as Code	4
7.1 Concepts	4
7.2 HCL syntax and a first project	4
7.3 Language features	5
7.4 Modules	6
7.5 State management in depth	7
7.6 Terraform in CI/CD and best practices	7
7.7 Terraform vs other IaC tools	7
8. Ansible: Configuration Management	9
8.1 Concepts	9
8.2 Setup and ad-hoc commands	9
8.3 Playbooks	10
8.4 Control flow, loops and variables	11
8.5 Roles	11
8.6 Secrets with Ansible Vault and dynamic inventory	12
8.7 Example: install Docker and run a container	12
8.8 Performance and best practices	12
9. Jenkins and CI/CD	14
9.1 CI/CD concepts	14
9.2 Jenkins architecture	14
9.3 Declarative Pipeline (Jenkinsfile)	15
9.4 Triggers and integrations	17
9.5 Scaling, security and operations	17
9.6 Jenkins vs other CI/CD tools	17
10. Related Topics	19
10.1 GitOps and ArgoCD	19
10.2 Monitoring, logging and observability	19
10.3 DevSecOps (security in the pipeline)	19
10.4 Artifact repositories and registries	20
10.5 Networking and web server basics for DevOps	20
10.6 YAML and JSON essentials	20
10.7 Service mesh and advanced Kubernetes ecosystem (awareness)	21
10.8 Release engineering and reliability practices	21
11. End-to-End Project: From Commit to Production	22
11.1 Architecture	22
11.2 Repository layout	22

11.3 Command sequence to practice	22
12. Quick Reference and Study Plan	23
12.1 One-page command cheat sheet	23
12.2 Concept comparison summary	23
12.3 A practical 8-week study plan	23
12.4 Frequently asked scenario questions	24

7. Terraform: Infrastructure as Code

7.1 Concepts

Terraform (HashiCorp; the open-source fork is **OpenTofu**) lets you describe cloud and SaaS infrastructure in declarative **HCL** files. You state *what* you want; Terraform computes *how* to get there by comparing your code, the **state file**, and the real world.

Benefits: repeatable environments, peer-reviewed changes in Git, drift detection, a preview of changes before applying (**plan**), and one workflow across AWS, Azure, GCP, Kubernetes, GitHub, Datadog and 3000+ providers.

Key terms

Term	Meaning
Provider	Plugin that talks to an API (aws, azure, google, kubernetes)
Resource	A managed object (EC2 instance, S3 bucket, VPC)
Data source	Read-only lookup of existing things (latest AMI, existing VPC)
Variable / Output	Inputs to and results from a module
Module	Reusable folder of Terraform code
State	JSON mapping of your config to real resource IDs (terraform.tfstate)
Backend	Where state is stored (local, S3, Azure Blob, Terraform Cloud)
Workspace	Multiple state instances from the same configuration

Workflow

```
terraform init      # download providers/modules, configure backend
terraform fmt -recursive # format code
terraform validate  # syntax and internal consistency check
terraform plan -out=tfplan # preview: + create, ~ update, - destroy, -/+ replace
terraform apply tfplan   # execute the saved plan (or apply without file, with prompt)
terraform destroy      # tear everything down
terraform output; terraform show; terraform state list
```

Never edit **terraform.tfstate** by hand and **never commit it to Git**: it can contain secrets. Use a remote backend.

7.2 HCL syntax and a first project

Layout: **main.tf**, **variables.tf**, **outputs.tf**, **providers.tf**, **terraform.tfvars**.

```
# providers.tf
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {
    # remote state with locking
    bucket      = "mycompany-tfstate"
    key         = "prod/network/terraform.tfstate"
    region      = "ap-south-1"
    dynamodb_table = "tf-locks" # prevents two applies at once
    encrypt     = true
  }
}
provider "aws" {
  region = var.region
  default_tags { tags = { Environment = var.env, ManagedBy = "terraform" } }
}
```

```
# variables.tf
variable "region"      { type = string, default = "ap-south-1" }
variable "env"         { type = string }
variable "instance_type" { type = string, default = "t3.micro" }
variable "allowed_cidrs" { type = list(string), default = ["0.0.0.0/0"] }
variable "db_password" { type = string, sensitive = true } # hidden in output

# main.tf
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]          # Canonical
  filter {
    name = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_security_group" "web" {
  name      = "${var.env}-web-sg"
  vpc_id    = module.vpc.vpc_id
  ingress {
    from_port = 80, to_port = 80, protocol = "tcp", cidr_blocks = var.allowed_cidrs
  }
  egress {
    from_port = 0, to_port = 0, protocol = "-1", cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "web" {
  count          = 2
  ami            = data.aws_ami.ubuntu.id
  instance_type  = var.instance_type
  subnet_id      = module.vpc.public_subnets[count.index]
  vpc_security_group_ids = [aws_security_group.web.id]
  user_data      = file("${path.module}/install_nginx.sh")
  tags           = { Name = "${var.env}-web-${count.index}" }

  lifecycle {
    create_before_destroy = true
  }
}

# outputs.tf
output "web_ips" {
  value     = aws_instance.web[*].public_ip
  description = "Public IPs of web servers"
}
```

Passing values: `terraform apply -var="env=dev", -var-file=prod.tfvars`, or environment variables `TF_VAR_env=dev`.

7.3 Language features

Types and expressions

Primitives: `string`, `number`, `bool`; collections: `list(...)`, `set(...)`, `map(...)`; structural: `object({...})`, `tuple([...])`. Reference resources as `aws_instance.web.id`, variables as `var.name`, locals as `local.name`, module outputs as `module.vpc.vpc_id`.

```

locals {
  name_prefix = "${var.env}-app"
  common_tags = { Project = "shop", Owner = "platform-team" }
}

# conditional
instance_type = var.env == "prod" ? "m5.large" : "t3.micro"

# for_each over a map: stable keys (better than count when items may be added/removed)
variable "buckets" { default = { logs = "private", assets = "public-read" } }
resource "aws_s3_bucket" "b" {
  for_each = var.buckets
  bucket   = "${local.name_prefix}-${each.key}"
}

# for expression and functions
output "upper_names" { value = [for n in var.names : upper(n) if n != ""] }
# useful functions: length, merge, lookup, join, split, format, element, flatten, toset, tomap, cidrsubn \
# et
cidr = cidrsubnet("10.0.0.0/16", 8, 1) # 10.0.1.0/24

# dynamic block: generate repeated nested blocks
dynamic "ingress" {
  for_each = var.ports
  content { from_port = ingress.value, to_port = ingress.value, protocol = "tcp", cidr_blocks = ["10.0.0 \
    .0/8"] }
}

```

count addresses resources by index (removing item 0 shifts the rest and can cause recreation); **for_each** addresses by key, which is safer.

Meta-arguments

depends_on (explicit dependency; implicit ones come from references), **count**, **for_each**, **provider** (alias for multiple regions/accounts), **lifecycle** (**create_before_destroy**, **prevent_destroy**, **ignore_changes**, **replace_triggered_by**), and **precondition/postcondition** checks.

7.4 Modules

A module is any folder with **.tf** files. The root module calls child modules to build reusable blocks.

```

module "vpc" {
  source = "terraform-aws-modules/vpc/aws" # registry module
  version = "~> 5.0"
  name    = "${var.env}-vpc"
  cidr    = "10.0.0.0/16"
  azs     = ["ap-south-1a", "ap-south-1b"]
  public_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  private_subnets = ["10.0.11.0/24", "10.0.12.0/24"]
  enable_nat_gateway = true
}

module "app" {
  source      = "../modules/app" # local module
  instance_type = "t3.small"
  subnet_ids   = module.vpc.private_subnets
}

```

Good module design: small focused inputs, sensible defaults, documented outputs, pinned versions, and no provider configuration inside child modules.

7.5 State management in depth

- **Remote backend + locking** gives a single source of truth and prevents concurrent applies. Enable bucket versioning and encryption.
- **Split state** by environment and by layer (network, data, app) to limit blast radius and speed up plans.

```
terraform state list
terraform state show aws_instance.web[0]
terraform state mv aws_instance.old aws_instance.new      # refactor without recreating
terraform state rm aws_instance.web[0]                    # stop managing (resource stays)
terraform import aws_s3_bucket.logs my-existing-bucket     # adopt an existing resource
terraform force-unlock <LOCK_ID>                          # only if a crashed run left a lock
terraform workspace new staging; terraform workspace select staging; terraform workspace list
terraform apply -replace=aws_instance.web[0]               # force recreate one resource
terraform plan -refresh-only                               # detect drift
```

Modern HCL also supports `moved { from = ..., to = ... }` and `import { to = ..., id = ... }` blocks so refactors and imports are reviewed in a plan like normal code.

Handling secrets

Mark variables `sensitive = true`, pull secrets from Secrets Manager/Vault data sources (note: values still land in state, so protect state), use short-lived credentials via OIDC in CI, and never commit `*.tfvars` containing secrets.

7.6 Terraform in CI/CD and best practices

- 1 Run `fmt -check`, `validate`, `tflint`, and security scanners (`tfsec/trivy config`, `checkov`) on each pull request.
- 2 Post `terraform plan` output in the PR; apply only from `main` after approval.
- 3 Pin provider and module versions; commit `.terraform.lock.hcl`.
- 4 Use separate accounts/state per environment (dev, staging, prod). **Terragrunt** helps keep multi-environment code DRY.
- 5 Tag everything and use consistent naming.
- 6 Policy as code: Sentinel or OPA to block risky changes (public buckets, oversized instances).

```
# GitHub Actions fragment
- uses: hashicorp/setup-terraform@v3
- run: terraform init -input=false
- run: terraform fmt -check && terraform validate
- run: terraform plan -input=false -out=tfplan
```

7.7 Terraform vs other IaC tools

	Terraform/OpenTofu	CloudFormation	Pulumi	Ansible
Language	HCL (declarative)	JSON/YAML	Python, TypeScript, Go, C#	YAML playbooks
Clouds	Multi-cloud, 3000+ providers	AWS only	Multi-cloud	Multi-cloud modules
State	Explicit state file	Managed by AWS stacks	State backend (managed or self)	Stateless
Primary purpose	Provisioning infrastructure	AWS provisioning	Provisioning in real languages	Configuration and orchestration
Strength	Ecosystem, plan/apply workflow	Deep AWS integration, rollback	Loops, tests, real code	Agentless, simple

Interview questions

- What is state and why remote? Maps config to real resources; remote gives sharing, locking, durability.
- `count` vs `for_each`? Index vs key identity; prefer `for_each` for stable addresses.
- What is drift and how do you fix it? Real infra differs from state; run `plan/refresh-only`, then apply to reconcile or update the code.
- `terraform taint` replacement? Deprecated; use `apply -replace=ADDRESS`.
- What happens if two people apply simultaneously? State locking (e.g. DynamoDB or native S3 locking) blocks the second.

8. Ansible: Configuration Management

8.1 Concepts

Ansible automates configuration, application deployment and orchestration. It is **agentless** (connects over SSH for Linux, WinRM for Windows, needs only Python on managed nodes), **declarative** and **idempotent**, and uses human-readable **YAML**. Key terms:

Term	Meaning
Control node	Machine where you run Ansible (Linux/macOS/WSL)
Managed nodes	Servers being configured
Inventory	List of hosts and groups
Module	Unit of work (apt , copy , service , user)
Task	One call to a module
Play	Maps a group of hosts to tasks
Playbook	YAML file with one or more plays
Role	Reusable, structured bundle of tasks, templates, vars, handlers
Handler	Task triggered by notify only when something changed (e.g. restart service)
Facts	Auto-discovered host data (ansible_os_family , ansible_default_ipv4)
Collection	Distribution format for roles, modules, plugins (amazon.aws , kubernetes.core)

8.2 Setup and ad-hoc commands

```
pip install ansible          # or: sudo apt install ansible
ansible --version
```

Inventory **inventory/hosts.ini**:

```
[web]
web1 ansible_host=10.0.1.11
web2 ansible_host=10.0.1.12

[db]
db1 ansible_host=10.0.2.21 ansible_user=ubuntu

[prod:children]
web
db

[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/prod.pem
```

Ad-hoc commands (quick one-liners):

```
ansible all -i inventory/hosts.ini -m ping
ansible web -m apt -a "name=nginx state=present update_cache=yes" --become
ansible db -m shell -a "df -h"
ansible web -m service -a "name=nginx state=restarted" -b
ansible all -m setup -a "filter=ansible_distribution*"          # show facts
ansible-inventory -i inventory/hosts.ini --graph
```

ansible.cfg in the project directory sets defaults:

```
[defaults]
inventory = inventory/hosts.ini
host_key_checking = False
forks = 20
[privilege_escalation]
become = True
```

8.3 Playbooks

```
# site.yml
---
- name: Configure web servers
  hosts: web
  become: true
  vars:
    app_port: 8080
    packages: [nginx, git, curl]
  tasks:
    - name: Install packages
      ansible.builtin.apt:
        name: "{{ packages }}"
        state: present
        update_cache: true
        when: ansible_os_family == "Debian"

    - name: Create app user
      ansible.builtin.user: { name: deploy, shell: /bin/bash, groups: sudo, append: true }

    - name: Deploy nginx config from template
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/conf.d/app.conf
        owner: root
        mode: "0644"
        validate: "nginx -t -c /etc/nginx/nginx.conf"
      notify: Reload nginx

    - name: Ensure nginx is running and enabled
      ansible.builtin.service: { name: nginx, state: started, enabled: true }

    - name: Wait for the app port
      ansible.builtin.wait_for: { port: 80, timeout: 30 }

  handlers:
    - name: Reload nginx
      ansible.builtin.service: { name: nginx, state: reloaded }
```

Run it:

```
ansible-playbook site.yml # run
ansible-playbook site.yml --check --diff # dry run showing what would change
ansible-playbook site.yml -l web1 -t config # limit to host, run only tagged tasks
ansible-playbook site.yml -e "app_port=9090" -v # extra vars, verbose (-vvv more)
ansible-playbook site.yml --syntax-check
ansible-lint site.yml # best-practice linting
```

Jinja2 template `templates/nginx.conf.j2`:

```
server {
    listen 80;
    server_name {{ inventory_hostname }};
    location / { proxy_pass http://127.0.0.1:{{ app_port }}; }
{% if enable_ssl | default(false) %}
    listen 443 ssl;
{% endif %}
}
```

8.4 Control flow, loops and variables

```
- name: Create several users
  ansible.builtin.user: { name: "{{ item.name }}", groups: "{{ item.groups }}" }
  loop:
    - { name: alice, groups: dev }
    - { name: bob, groups: ops }

- name: Run a command and capture the result
  ansible.builtin.command: cat /etc/os-release
  register: osinfo
  changed_when: false           # read-only command should not report "changed"

- ansible.builtin.debug: { msg: "Running {{ osinfo.stdout_lines[0] }}" }

- name: Only on production
  ansible.builtin.copy: { src: prod.env, dest: /opt/app/.env, mode: "0600" }
  when: env == "prod" and inventory_hostname in groups['web']

- name: Retry until healthy
  ansible.builtin.uri: { url: "http://localhost:8080/health", status_code: 200 }
  register: result
  until: result.status == 200
  retries: 10
  delay: 5

- block:
  - ansible.builtin.command: /opt/app/migrate.sh
  rescue:
  - ansible.builtin.debug: { msg: "Migration failed, rolling back" }
  always:
  - ansible.builtin.debug: { msg: "Done" }
```

Variable precedence (low to high, simplified): role defaults < inventory vars < group_vars < host_vars < play vars < vars_files < role vars < task vars < **--extra-vars** (always wins). Put environment data in **group_vars/prod.yml** and **host_vars/web1.yml**.

8.5 Roles

Roles give structure and reuse. Create with **ansible-galaxy init roles/nginx**:

```
roles/nginx/
  tasks/main.yml      # the work
  handlers/main.yml   # restart/reload handlers
  templates/          # .j2 files
  files/              # static files
  defaults/main.yml   # lowest precedence variables (easy to override)
  vars/main.yml       # higher precedence variables
  meta/main.yml       # dependencies and metadata
```

Use in a playbook, and install community roles/collections from Galaxy:

```
- hosts: web
  become: true
  roles:
    - common
    - { role: nginx, app_port: 8080 }
```

```
ansible-galaxy collection install amazon.aws kubernetes.core
ansible-galaxy install geerlingguy.docker
```

8.6 Secrets with Ansible Vault and dynamic inventory

```
ansible-vault create group_vars/prod/vault.yml      # encrypted file
ansible-vault edit group_vars/prod/vault.yml
ansible-vault encrypt_string 'S3cr3t!' --name 'db_password' # inline encrypted value
ansible-playbook site.yml --ask-vault-pass          # or --vault-password-file ~/.vault_pass
```

Dynamic inventory discovers hosts from the cloud instead of a static file, for example the `amazon.aws.aws_ec2` plugin:

```
# inventory/aws_ec2.yml
plugin: amazon.aws.aws_ec2
regions: [ap-south-1]
filters: { tag:Environment: prod, instance-state-name: running }
keyed_groups: [{ key: tags.Role, prefix: role }]
```

8.7 Example: install Docker and run a container

```
- hosts: app
  become: true
  tasks:
    - name: Install Docker prerequisites
      ansible.builtin.apt: { name: [ca-certificates, curl, docker.io, python3-docker], state: present, update_cache: true }
    - name: Add user to docker group
      ansible.builtin.user: { name: ubuntu, groups: docker, append: true }
    - name: Start Docker
      ansible.builtin.service: { name: docker, state: started, enabled: true }
    - name: Run app container
      community.docker.docker_container:
        name: web
        image: myrepo/web:1.4.2
        state: started
        restart_policy: unless-stopped
        published_ports: ["80:8080"]
        env: { DB_HOST: "{{ db_host }}" }
```

8.8 Performance and best practices

- Keep playbooks **idempotent**: prefer modules (`apt`, `lineinfile`, `template`) over `shell/command`; if you must use them, add `creates:`, `when:` or `changed_when`.
- Use `strategy: free`, increase `forks`, enable SSH **pipelining** and `gather_facts: false` when facts are not needed.
- **Rolling updates**: `serial: 2` (or "25%") updates a few hosts at a time; `max_fail_percentage` aborts on failures.
- Use tags, `--check`, `ansible-lint`, and test roles with **Molecule**.
- **AWX / Ansible Automation Platform** adds web UI, RBAC, scheduling and audit.

Ansible vs Terraform vs Chef/Puppet

	Ansible	Terraform	Chef / Puppet
Main job	Configure servers, deploy apps, orchestrate	Provision infrastructure	Configure servers
Style	Mostly procedural tasks in order, with idempotent modules	Declarative with state	Declarative (Puppet) / Ruby DSL (Chef)
Agent	None (SSH)	None (API calls)	Agent required
State	Stateless	State file	Server-held catalogs
Learning curve	Easy	Moderate	Steeper

Typical combination: **Terraform creates** the VMs, networks and clusters; **Ansible configures** the software inside them (or Terraform outputs feed Ansible's inventory). With container platforms much configuration shifts into images and Kubernetes manifests, but Ansible remains excellent for VMs, bare metal, network devices and glue automation.

Interview questions

- What does idempotent mean? Re-running yields no further change if the system is already in the desired state.
- Handler vs task? A handler runs only if notified by a changed task, once at the end of the play.
- `command` vs `shell`? `shell` supports pipes/redirects through a shell; `command` does not and is safer.
- How do you encrypt passwords? Ansible Vault.
- How does Ansible connect? SSH (Linux) or WinRM (Windows); no agents.

9. Jenkins and CI/CD

9.1 CI/CD concepts

- **Continuous Integration (CI):** developers merge to a shared branch frequently; each change is automatically built and tested, so integration bugs surface within minutes.
- **Continuous Delivery:** every passing build produces a releasable artifact, deployed to staging automatically; production release needs a manual approval.
- **Continuous Deployment:** production release is also automatic when all checks pass.
- **Pipeline:** ordered stages such as Checkout -> Build -> Unit test -> Static analysis -> Package -> Image build/scan -> Push -> Deploy dev -> Integration tests -> Approval -> Deploy prod.
- **Artifact:** versioned output of a build (jar, npm package, Docker image) stored in a repository (Nexus, Artifactory, ECR).
- **Quality gates:** SonarQube, test coverage thresholds and vulnerability scans that fail the build when standards are not met.

9.2 Jenkins architecture

Jenkins is an open-source automation server written in Java with a huge plugin ecosystem.

Component	Role
Controller (master)	Hosts UI, schedules jobs, stores config. Should run few or no builds itself
Agent (node)	Executes build steps; connected via SSH, JNLP/WebSocket, Docker or Kubernetes pods
Executor	A slot on an agent running one build at a time
Job / Project	A configured automation (Freestyle, Pipeline, Multibranch, Folder)
Plugin	Extension: Git, Pipeline, Docker, Kubernetes, Credentials, Blue Ocean
Workspace	Directory where a build runs
JENKINS_HOME	Config, jobs and plugin data (back it up); default <code>/var/lib/jenkins</code>

Installing (Ubuntu or Docker)

```
# Docker (quick start); needs Java 17+ if installing directly on a host
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts-jdk17
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword # first login password
# Package install on Ubuntu (after adding the Jenkins apt repo + key)
sudo apt install -y fontconfig openjdk-17-jre jenkins
sudo systemctl enable --now jenkins # UI on port 8080
```

After the setup wizard: install suggested plugins, create an admin user, configure the URL, then add credentials and agents.

9.3 Declarative Pipeline (Jenkinsfile)

Pipelines as code live in a **Jenkinsfile** in the repository root, so build logic is versioned and reviewed.

```

pipeline {
  agent any                                // or: agent { label 'linux-docker' }
  options {
    timeout(time: 30, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '20'))
    disableConcurrentBuilds()
    timestamps()
  }
  environment {
    IMAGE = "myrepo/web"
    TAG   = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
    REGISTRY_CREDS = credentials('dockerhub-creds') // username/password credential
  }
  parameters {
    choice(name: 'DEPLOY_ENV', choices: ['dev', 'staging', 'prod'], description: 'Target')
    booleanParam(name: 'SKIP_TESTS', defaultValue: false)
  }
  triggers { pollSCM('H/5 * * * *') } // webhooks are better than polling

  stages {
    stage('Checkout') { steps { checkout scm } }

    stage('Build and Test') {
      parallel {
        stage('Unit tests') {
          when { expression { !params.SKIP_TESTS } }
          steps { sh 'npm ci && npm test' }
          post { always { junit 'reports/**/*.xml' } }
        }
        stage('Lint') { steps { sh 'npm run lint' } }
      }
    }

    stage('SonarQube') {
      steps {
        withSonarQubeEnv('sonar') { sh 'sonar-scanner' }
        timeout(time: 5, unit: 'MINUTES') { waitForQualityGate abortPipeline: true }
      }
    }

    stage('Docker build and push') {
      steps {
        sh '''
          docker build -t $IMAGE:$TAG .
          echo "$REGISTRY_CREDS_PSW" | docker login -u "$REGISTRY_CREDS_USR" --password-stdin
          docker push $IMAGE:$TAG
        '''
      }
    }
  }
}

```

```

    ...
  }
}

stage('Scan image') { steps { sh 'trivy image --exit-code 1 --severity HIGH,CRITICAL $IMAGE:$TAG' } \
}

stage('Deploy') {
  when { branch 'main' }
  steps {
    input message: "Deploy ${TAG} to ${params.DEPLOY_ENV}?", ok: 'Deploy'
    withCredentials([file(credentialsId: 'kubeconfig', variable: 'KUBECONFIG')]) {
      sh 'kubectl set image deployment/web web=$IMAGE:$TAG -n ${DEPLOY_ENV}'
      sh 'kubectl rollout status deployment/web -n ${DEPLOY_ENV} --timeout=120s'
    }
  }
}
}

post {
  success { echo 'Pipeline succeeded' }
  failure { mail to: 'team@company.com', subject: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}", body \
: "${env.BUILD_URL}" }
  always { cleanWs() }
}
}

```

Declarative vs Scripted pipeline

	Declarative	Scripted
Syntax	Structured: <code>pipeline { stages { stage { steps } } }</code>	Groovy: <code>node { stage('x') { ... } }</code>
Learning curve	Easier, validated up front	Flexible, full Groovy power
Error handling	<code>post { }</code> blocks	<code>try/catch/finally</code>
Recommended	Yes for most cases	Complex dynamic logic only

Useful building blocks

- **Agents:** `agent { docker { image 'node:20-alpine' } }` runs stages inside a container for clean, reproducible builds. `agent { kubernetes { yamlFile 'pod.yaml' } }` spins up a pod per build (Kubernetes plugin).
- **Credentials:** stored encrypted in Jenkins (**Manage Jenkins -> Credentials**): username/password, secret text, SSH key, file, certificates. Always use `credentials()` or `withCredentials`; Jenkins masks secrets in logs.
- **Multibranch pipelines** scan a repository, create a job for every branch and pull request containing a Jenkinsfile.
- **Shared libraries** keep reusable Groovy functions in a separate repo: `@Library('my-shared-lib')` _ then `deployApp(env: 'prod')`.
- **Stash/unstash and archiveArtifacts** pass files between stages/agents or keep build outputs.
- **when conditions:** `branch`, `tag`, `changeRequest`, `environment`, `expression`, `anyOf/allOf`.
- `retry(3)`, `timeout`, `catchError` for resilience.

9.4 Triggers and integrations

Trigger	How
Webhook (best)	GitHub/GitLab sends HTTP POST to <code>https://jenkins/github-webhook/</code> on push or PR
Poll SCM	Jenkins checks the repo on a cron schedule (<code>H/5 * * * *</code>)
Scheduled	<code>cron('H 2 * * *')</code> nightly builds
Upstream	Run after another job finishes
Manual / API	Build with parameters, <code>curl -X POST -u user:token https://jenkins/job/x/build</code>

The **H** in cron spreads load by hashing the job name.

9.5 Scaling, security and operations

- **Use agents:** run builds on ephemeral agents (Docker, Kubernetes, EC2 spot fleet) so the controller stays healthy and builds are isolated and clean.
- **Security:** enable authentication (LDAP/SSO/OIDC), role-based authorization (Role Strategy plugin), CSRF protection, restrict script approvals, keep Jenkins and plugins updated, never put secrets in the Jenkinsfile, run behind HTTPS reverse proxy.
- **Configuration as Code (JCasC)** plugin defines Jenkins settings in YAML; use it with plugins lists for reproducible controllers.
- **Backups:** snapshot `JENKINS_HOME`, or use ThinBackup plugin. Monitor with the Prometheus plugin.
- **Common issues:** out of disk (log rotation, `cleanws`), stuck builds (timeouts), plugin conflicts (update in staging first), permission denied for Docker (add `jenkins` user to `docker` group), slow polling (use webhooks).

9.6 Jenkins vs other CI/CD tools

	Jenkins	GitHub Actions	GitLab CI	CircleCI / Azure DevOps
Hosting	Self-hosted (you operate it)	SaaS (and self-hosted runners)	SaaS or self-managed	SaaS
Config	Jenkinsfile (Groovy)	YAML workflows in <code>.github/workflows</code>	<code>.gitlab-ci.yml</code>	YAML
Plugins	1800+, flexible but need upkeep	Marketplace of actions	Built-in features	Orbs / extensions
Setup effort	Highest	Lowest	Low	Low
Best for	Complex/legacy pipelines, on-prem, full control	GitHub-based projects	All-in-one DevOps platform	Managed convenience

Equivalent GitHub Actions workflow:

```
# .github/workflows/ci.yml
name: ci
on: { push: { branches: [main] }, pull_request: {} }
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci && npm test
      - uses: docker/login-action@v3
        with: { username: "${{ secrets.DOCKER_USER }}", password: "${{ secrets.DOCKER_TOKEN }}" }
      - uses: docker/build-push-action@v6
        with: { push: true, tags: "myrepo/web:${{ github.sha }}" }
```

GitLab CI equivalent: `stages: [build, test, deploy]` with jobs that define `stage`, `image`, `script`, `only/rules`.

Interview questions

- Freestyle vs Pipeline job? Pipeline is code in a Jenkinsfile with stages, versioned and durable; Freestyle is UI-configured.
- How do you secure credentials? Jenkins Credentials store + `withCredentials`, folder-scoped access, external vaults.
- Controller vs agent? Controller orchestrates; agents run the work.
- How do you trigger a build on push? Webhook from Git server to Jenkins.
- How do you handle build failures? `post { failure {} }` notifications, retries, timeouts, quality gates.

10. Related Topics

10.1 GitOps and ArgoCD

GitOps: Git is the single source of truth for the desired state of the cluster; an in-cluster agent (ArgoCD or Flux) continuously pulls from Git and reconciles. Benefits: audit trail via commits, easy rollback by reverting, no cluster credentials stored in CI (pull instead of push), drift is auto-corrected.

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: web, namespace: argocd }
spec:
  project: default
  source: { repoURL: https://github.com/org/deploy-repo.git, targetRevision: main, path: apps/web/overlays/prod }
  destination: { server: https://kubernetes.default.svc, namespace: prod }
  syncPolicy: { automated: { prune: true, selfHeal: true }, syncOptions: [CreateNamespace=true] }
```

Typical flow: Jenkins/Actions builds and pushes an image, then updates the image tag in the deployment repo; ArgoCD detects the commit and syncs the cluster.

10.2 Monitoring, logging and observability

Pillar	Question	Tools
Metrics	How is it performing? (numbers over time)	Prometheus, Grafana, CloudWatch, Datadog
Logs	What happened?	ELK/EFK (Elasticsearch, Fluentd/Fluent Bit, Kibana), Loki, CloudWatch Logs
Traces	Where did the request spend time across services?	Jaeger, Tempo, OpenTelemetry, X-Ray

Prometheus scrapes `/metrics` endpoints on a pull model, stores time series, and evaluates alert rules (Alertmanager routes to Slack, PagerDuty, email). Query with **PromQL**:

```
rate(http_requests_total{status!="5.."}[5m]) # error rate per second
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le)) # p95 latency
100 - (avg by(instance)(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) # CPU %
```

Install on Kubernetes with `helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack`.

- **Golden signals** (Google SRE): latency, traffic, errors, saturation. **SLI** (measurement), **SLO** (target, e.g. 99.9% success), **SLA** (contractual promise), **error budget** (allowed unreliability).
- Alert on symptoms users feel, not every cause; every alert needs an owner and a runbook.

10.3 DevSecOps (security in the pipeline)

Practice	Tools
Secret scanning	gitLeaks, trufflehog, GitHub secret scanning
SAST (static code analysis)	SonarQube, Semgrep, CodeQL
Dependency scanning (SCA)	Dependabot, Snyk, OWASP Dependency-Check
Container image scanning	Trivy, Grype, ECR scanning
IaC scanning	tfsec, Checkov, KICS
DAST (running app tests)	OWASP ZAP
Runtime security	Falco, admission controllers (Kyverno, OPA Gatekeeper)
Secrets management	HashiCorp Vault, AWS Secrets Manager, External Secrets Operator

Practice	Tools
Supply chain	SBOMs (Syft), image signing (cosign), SLSA provenance

10.4 Artifact repositories and registries

Nexus and JFrog Artifactory store jars, npm, pip and Docker artifacts; cloud registries (ECR, ACR, GAR, GHCR) store images. Use immutable tags (git SHA or semantic version), retention policies and vulnerability scanning on push.

10.5 Networking and web server basics for DevOps

- **DNS** maps names to IPs (A, AAAA, CNAME, MX, TXT records); TTL controls caching.
- **HTTP**: methods (GET, POST...), status codes (2xx ok, 3xx redirect, 4xx client error, 5xx server error), headers, TLS handshake and certificates (Let's Encrypt via certbot or cert-manager).
- **Reverse proxy and load balancer**: Nginx example:

```
upstream app { server 10.0.1.11:8080; server 10.0.1.12:8080; }
server {
    listen 80;
    server_name app.example.com;
    location / {
        proxy_pass http://app;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

- **CIDR quick math**: $/32 = 1$ address, $/24 = 256$, $/16 = 65,536$. Private ranges: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`.
- **Layer 4 vs Layer 7**: TCP/UDP-level balancing vs HTTP-aware routing.

10.6 YAML and JSON essentials

YAML is whitespace sensitive (spaces only, never tabs) and is the language of Kubernetes, Ansible, Compose and CI files.

```
name: demo                # scalar
ports: [80, 443]          # inline list
env:                      # map
  LOG_LEVEL: info
  FEATURES:
    - search              # block list
    - export
script: |                 # multi-line, keeps newlines
  echo one
  echo two
base: &base { retries: 3 } # anchor
job: { <<: *base, timeout: 30 } # merge with alias
```

Validate with `yamllint`, `kubectl apply --dry-run=client`, or `yq`. Quote strings like "yes", "on", "1.0" to avoid unintended type conversion.

10.7 Service mesh and advanced Kubernetes ecosystem (awareness)

Istio/Linkerd add mTLS, traffic shaping, retries and telemetry between services using sidecar proxies. **Operators** extend Kubernetes with custom resources to manage complex software (Postgres, Kafka). **Argo Rollouts/Flagger** automate canary and blue/green. **KEDA** autoscales on events (queue length). **Crossplane** manages cloud infrastructure through Kubernetes APIs.

10.8 Release engineering and reliability practices

- **Semantic versioning** **MAJOR.MINOR.PATCH**; tag releases in Git, build once and promote the *same* artifact through environments.
- **Feature flags** decouple deploy from release; **database migrations** (Flyway, Liquibase) should be backward compatible (expand, migrate, contract).
- **Rollback plan** for every release; practice it.
- **Incident management**: detect, triage, mitigate, communicate, resolve, then a blameless post-mortem with action items.
- **Chaos engineering** (Chaos Monkey, Litmus) deliberately injects failure to prove resilience.
- **DORA metrics** measure delivery performance: deployment frequency, lead time for changes, change failure rate, time to restore service.

11. End-to-End Project: From Commit to Production

11.1 Architecture

A developer pushes code to **GitHub**. A webhook triggers **Jenkins**, which builds and tests the app, builds a **Docker image**, scans it, pushes it to **ECR**, and updates the deployment. The infrastructure (VPC, EKS cluster, ECR, RDS) is created by **Terraform**; the supporting VMs (Jenkins server, bastion) are configured by **Ansible**; the app runs on **Kubernetes** and is observed with **Prometheus and Grafana**.

Step	Tool	Action
1	Terraform	<code>terraform apply</code> creates VPC, subnets, EKS, ECR, IAM roles, RDS
2	Ansible	Installs and configures Jenkins, Docker, kubectl, trivy on the CI VM
3	Git	Developer opens PR; checks run; merge to <code>main</code>
4	Jenkins	Build, test, SonarQube gate, docker build, Trivy scan, push image tagged with commit SHA
5	Kubernetes	<code>kubectl set image</code> or Helm upgrade performs a rolling update; readiness probes gate traffic
6	Monitoring	Prometheus alerts on error rate and latency; Grafana dashboards; logs to Loki

11.2 Repository layout

```
repo/
  app/                # source code + Dockerfile
  Jenkinsfile
  k8s/ or helm/       # manifests or chart
  infra/terraform/
    envs/dev  envs/prod # per-environment root modules
    modules/vpc modules/eks
  ansible/
    inventory/ roles/jenkins roles/docker site.yml
```

11.3 Command sequence to practice

```
# 1. Infrastructure
cd infra/terraform/envs/dev && terraform init && terraform plan -out=tf.plan && terraform apply tf.plan
aws eks update-kubeconfig --name dev-cluster --region ap-south-1

# 2. Configure CI server
cd ansible && ansible-playbook -i inventory/aws_ec2.yml site.yml --limit ci

# 3. Build and ship manually once (what Jenkins will automate)
docker build -t $ECR/web:v1 app/ && docker push $ECR/web:v1
helm upgrade --install web helm/web -n dev --create-namespace --set image.tag=v1

# 4. Verify
kubectl rollout status deploy/web -n dev && kubectl get ingress -n dev
curl -I https://dev.example.com/health
```

12. Quick Reference and Study Plan

12.1 One-page command cheat sheet

Tool	Most-used commands
Linux	<code>ls -lah, grep -rn, find, tail -f, ps aux, top, df -h, du -sh, ss -tulpn, chmod/chown, systemctl, journalctl -u, curl -v</code>
Git	<code>status, add -p, commit, switch -c, merge/rebase, log --oneline --graph, stash, reset/revert, reflog, cherry-pick</code>
Docker	<code>build, run -d -p, ps, logs -f, exec -it, compose up -d, system prune, push</code>
Kubernetes	<code>get/describe/logs/exec, apply -f, rollout status/undo, scale, port-forward, top, config use-context</code>
Terraform	<code>init, fmt, validate, plan, apply, destroy, state list/mv/rm, import, workspace</code>
Ansible	<code>ansible -m ping, ansible-playbook --check --diff, -l, -t, ansible-vault, ansible-galaxy</code>
Jenkins	<code>Jenkinsfile</code> stages, <code>credentials()</code> , <code>withCredentials</code> , <code>parallel</code> , <code>post</code> , <code>input</code> , shared libraries
AWS	<code>sts get-caller-identity, ec2 describe-instances, s3 sync, eks update-kubeconfig</code>

12.2 Concept comparison summary

Question	Answer
Docker vs VM	Containers share a kernel (light, fast); VMs run full OSes (strong isolation)
Terraform vs Ansible	Provision infrastructure vs configure what runs on it
Pod vs Deployment	One running unit vs a controller managing replicas and rollouts
Jenkins vs GitHub Actions	Self-hosted flexible engine vs hosted, repo-native workflows
Push vs pull deployment	CI pushes to cluster vs GitOps agent pulls from Git
Mutable vs immutable infra	Patch servers in place vs replace with new images
Blue/green vs canary	Switch all traffic between two environments vs shift a small percentage gradually
Scale up vs out	Bigger machine vs more machines
RPO vs RTO	Acceptable data loss vs acceptable downtime

12.3 A practical 8-week study plan

- Week 1:** Linux commands, permissions, processes, shell scripts, SSH; practice on a free VM or WSL.
- Week 2:** Git workflows, branching, PRs, resolving conflicts; contribute to a small repo with a teammate.
- Week 3:** Cloud basics (AWS free tier): EC2, S3, IAM, VPC, load balancer, billing alarms.
- Week 4:** Docker: write Dockerfiles, multi-stage builds, Compose a 3-tier app.
- Week 5:** Kubernetes: kind/minikube, Deployments, Services, Ingress, ConfigMaps, probes, HPA, Helm.
- Week 6:** Terraform: VPC + EC2 + S3 with remote state, modules, workspaces; then an EKS cluster.
- Week 7:** Ansible: roles, templates, vault, dynamic inventory; configure the Jenkins server with it.
- Week 8:** Jenkins pipeline end to end plus monitoring, then rebuild the whole project from scratch without notes.

Tip: do not just read. Break things on purpose (kill a pod, fill a disk, misconfigure a security group) and practice diagnosing them. Keep every lab in Git, destroy cloud resources afterwards (`terraform destroy`) to avoid bills, and explain each tool aloud as if teaching someone: that is what interviews test.

12.4 Frequently asked scenario questions

- **A deployment is stuck at 0 pods ready. What do you check?** `kubectl describe` for events, image pull errors, probe failures, resource requests vs node capacity, Secrets/ConfigMaps present, then logs.
- **The site is down after a release.** Check monitoring and recent deploys; `kubectl rollout undo` to restore service first, investigate afterwards.
- **How would you design CI/CD for a microservices app?** One pipeline per service from Git, build once, immutable image tags, automated tests and scans, deploy via Helm/GitOps to dev -> staging -> prod with approvals, observability and automated rollback on failed health checks.
- **How do you manage secrets across the toolchain?** Never in Git; vault/secrets manager, short-lived OIDC credentials, Jenkins credentials store, Kubernetes External Secrets, encrypted Ansible Vault and sensitive Terraform variables with protected state.
- **How do you reduce cloud cost?** Tagging and budgets, right-sizing, auto-scaling, Spot for CI/batch, schedules to turn off non-prod, storage lifecycle policies, Savings Plans for steady load.